

Politechnika Warszawska

WYDZIAŁ ELEKTRONIKI
I TECHNIK INFORMACYJNYCH



Instytut Mikroelektroniki i Optoelektroniki

Praca dyplomowa inżynierska

na kierunku Elektronika
w specjalności Elektronika i Fotonika

Dedykowana aplikacja do analizy analogowych układów elektronicznych
w języku programowania Swift

Oleksandr Bolbat

Numer albumu 329915

promotor
mgr inż. Bartosz Dec

WARSZAWA 2026

Streszczenie

Praca inżynierska skupia się na napisaniu dedykowanej aplikacji do analizy analogowych układów elektronicznych w języku programowania Swift. Aplikacja stanowi próbę pokazania możliwości tworzenia natywnych narzędzi do analizy układów elektronicznych dla systemu macOS. Prace rozpoczęto od zebrania informacji dotyczących sposobów tworzenia aplikacji służących do analizy układów elektronicznych, metod numerycznych pozwalających na ich analizę oraz pogłębienia znajomości języka programowania Swift. Zakres pracy obejmował napisanie poszczególnych bloków aplikacji umożliwiających parsowanie opisu układu ze zwykłego pliku tekstowego, tworzenie elementów na podstawie ich szablonów, obliczanie punktu pracy i analizę czasową układu przy użyciu metod numerycznych. W procesie zostały napisane zabezpieczenia chroniące przed analizowaniem błędnych układów oraz testy sprawdzające poszczególne bloki aplikacji. Poprawność opracowanego rozwiązania została sprawdzona na prostych układach oraz na przykładzie wtórника emiterowego.

Słowa kluczowe: ZMPW, symulacja obwodów, SPICE, Swift, metody numeryczne.

Abstract

The engineering thesis focuses on developing a dedicated application for the analysis of analog electronic circuits in the Swift programming language. The application is intended to demonstrate the possibility of creating native circuit analysis tools for macOS. The work began with gathering information on applications used for electronic circuit analysis, numerical methods applied to such analysis, and the Swift programming language. The scope of the work included implementing the main application modules responsible for parsing a circuit description from a plain text file, creating elements based on their templates, calculating the operating point, and performing time-domain analysis using numerical methods. During the development process, safeguards were implemented to prevent the analysis of incorrect circuits, and tests were prepared to verify individual application modules. The correctness of the developed solution was checked on simple circuits and on an emitter follower example.

Keywords: MNA, circuit simulation, SPICE, Swift, numerical methods.

Spis treści

Wykaz symboli i skrótów	1
Spis rysunków	2
Spis tabel	2
1 Wstęp	3
1.1 Motywacja	3
1.2 Cel pracy	3
1.3 Zakres pracy	4
1.4 Struktura pracy	4
2 Podstawy teoretyczne	5
2.1 Równania obwodów	5
2.2 Metody macierzowe w analizie obwodów	6
2.3 Klasyczna metoda potencjałów węzłowych	7
2.4 Zmodyfikowana metoda potencjałów węzłowych	7
2.5 Punkt pracy DC	8
2.6 Analiza czasowa	8
3 Zmodyfikowana metoda potencjałów węzłowych i stemple elementów	10
3.1 Rola stempli w metodzie ZMPW	10
3.2 Ogólna postać stempla	10
3.3 Stempel rezystora	11
3.4 Stemple źródeł niezależnych	11
3.5 Stemple źródeł sterowanych	12
3.6 Stemple elementów nieliniowych	13
3.7 Wypełnianie macierzy globalnej	13
3.8 Znaczenie dodatkowych wierszy	14
4 Założenia i architektura aplikacji	15
4.1 Założenia projektowe	15

4.2	Ogólna architektura	15
4.3	Format pliku wejściowego	17
4.4	Parser netlisty	19
4.5	Przypisywanie dodatkowych równań	19
4.6	Reprezentacja elementów i stempli	20
4.7	Solver i komendy analizy	20
4.8	Obsługa błędów	22
5	Implementacja	24
5.1	Struktura kodu	24
5.2	Parser i wynik parsowania	24
5.3	Tworzenie stempli	25
5.4	Macierze	26
5.5	Solver	27
5.6	Interfejs użytkownika i wizualizacja wyników	28
5.7	Obsługa błędów w implementacji	28
5.8	Uwagi implementacyjne	29
6	Testy	30
6.1	Cel testowania	30
6.2	Zakres testów	30
6.3	Testy parsera	31
6.4	Testy stempli i elementów	32
6.5	Testy błędów stempli	32
6.6	Testy solvera i macierzy	33
6.7	Testy wizualizacji	33
6.8	Znaczenie testów dla rozwoju programu	33
7	Podsumowanie	34
7.1	Wnioski	34
7.2	Ocena przyjętego podejścia	35
7.3	Możliwe kierunki rozwoju	35
7.4	Zakończenie	36
	Bibliografia	37

Wykaz symboli i skrótów

ECAP Electronic Circuit Analysis Program.

SPICE Simulation Program with Integrated Circuit Emphasis; oprogramowanie do modelowania oraz symulacji działania elektronicznych układów analogowych i cyfrowych.

DC Direct Current; prąd stały albo analiza stałoprądowa.

AC Alternating Current; prąd przemienny albo wymuszenie sinusoidalne.

ZMPW Zmodyfikowana Metoda Potencjałów Węzłowych; metoda analizy obwodów elektrycznych oparta na prawach Kirchhoffa i charakterystykach napięciowo-prądowych elementów.

KCL Kirchhoff Current Law; pierwsze prawo Kirchhoffa, czyli prądowe prawo Kirchhoffa.

KVL Kirchhoff Voltage Law; drugie prawo Kirchhoffa, czyli napięciowe prawo Kirchhoffa.

BJT Bipolar Junction Transistor; tranzystor bipolarny.

UI User Interface; interfejs użytkownika.

LAPACK Linear Algebra Package; biblioteka procedur numerycznych algebry liniowej.

UTF-8 Unicode Transformation Format 8-bit; kodowanie tekstu używane przez parser.

UTI Uniform Type Identifier; systemowy identyfikator typu danych używany w macOS.

NaN Not a Number; liczba o niezdefiniowanej lub niereprezentowalnej wartości.

Inf Infinity; reprezentacja nieskończoności.

N-R Metoda Newtona-Raphsona; algorytm iteracyjny prowadzący do wyznaczenia przybliżonej wartości miejsca zerowego funkcji jednej lub wielu zmiennych.

Spis rysunków

4.1	Główne okno aplikacji z edytorem opisu układu	17
4.2	Wynik analizy punktu pracy dla dzielnika napięcia	21
4.3	Wynik analizy czasowej prostego układu ze źródłem sinusoidalnym i rezystorem	21
4.4	Wynik symulacji czasowej wtórniaka emiterowego	22

Spis tabel

4.1	Format opisu obsługiwanych elementów	18
6.1	Główne grupy testów	31

Rozdział 1

Wstęp

1.1 Motywacja

Analiza układów elektronicznych przez wiele lat była wykonywana ręcznie, na podstawie równań obwodowych i obliczeń prowadzonych przez projektantów. Taki sposób pracy był czasochłonny, podatny na błędy i trudny do stosowania w przypadku bardziej rozbudowanych układów. Rozwój komputerów doprowadził do powstania programów wspomagających analizę obwodów, takich jak ECAP oraz SPICE, które stały się podstawą współczesnych symulatorów układów analogowych [1], [2], [3].

Wraz z rozwojem narzędzi symulacyjnych wzrosła także ich złożoność oraz zależność od określonych platform systemowych. Wiele popularnych programów inżynierskich jest projektowanych przede wszystkim z myślą o systemie Windows albo wymaga na macOS rozwiązań pośrednich, takich jak wirtualizacja. Jednocześnie komputery Apple, szczególnie oparte na układach Apple Silicon, stanowią wydajną platformę do obliczeń numerycznych i tworzenia natywnych aplikacji inżynierskich.

Z tego powodu w pracy podjęto próbę zaprojektowania i zaimplementowania aplikacji do analizy analogowych układów elektronicznych dla systemu macOS. Program opiera się na tekstowym opisie układu, metodzie Modified Nodal Analysis oraz mechanizmie stempli elementów, które pozwalają budować globalny układ równań w sposób modularny [4], [5].

1.2 Cel pracy

Celem pracy jest zaprojektowanie i zaimplementowanie programu, który wczytuje tekstowy opis układu, przekształca elementy obwodu na stemple macierzowe oraz wykonuje podstawowe analizy numeryczne układu. Istotnym założeniem projektu jest zachowanie przejrzystej architektury, w której parser, modele elementów, macierze, solver punktu pracy oraz moduł analizy czasowej stanowią oddzielne bloki programu. Implementacja została wykonana w języku Swift, który udostępnia statyczne typowanie, mechanizmy obsługi błędów oraz narzędzia do budowy

natywnych aplikacji dla systemu macOS [6].

1.3 Zakres pracy

Zakres implementacji obejmuje parser plików tekstowych, model stempli podstawowych elementów liniowych, składanie globalnej macierzy układu, solver dla komendy punktu pracy oraz realizację analizy czasowej. Program obsługuje rezystory, niezależne źródła prądowe i napięciowe oraz podstawowe źródła sterowane. Analiza czasowa rozszerza działanie programu o przetwarzanie odpowiedzi układu w kolejnych krokach czasu.

1.4 Struktura pracy

W rozdział 2 przedstawiono podstawowe pojęcia teorii obwodów i symulacji komputerowej. Rozdział 3 opisuje metodę Modified Nodal Analysis oraz ideę stempli elementów. Rozdział 4 i rozdział 5 omawiają architekturę oraz implementację programu. Rozdział 6 przedstawia sposób weryfikacji działania aplikacji, a rozdział 7 zawiera wnioski i możliwe kierunki dalszego rozwoju.

Rozdział 2

Podstawy teoretyczne

2.1 Równania obwodów

Podstawą komputerowej analizy obwodów jest zapis zależności pomiędzy prądami i napięciami w postaci równań matematycznych. Obwód można przedstawić jako zbiór elementów połączonych węzłami. Węzeł jest punktem o wspólnym potencjale elektrycznym, natomiast element narzuca zależność między prądem płynącym przez jego zaciski a napięciem na tych zaciskach.

Najważniejszymi prawami wykorzystywanymi do tworzenia równań obwodowych są prawa Kirchhoffa. Wynikają one z zasad zachowania ładunku i energii, dlatego stanowią podstawę zarówno ręcznej analizy obwodów, jak i algorytmów stosowanych w symulatorach typu SPICE [5], [7].

Pierwsze prawo Kirchhoffa, nazywane prądowym prawem Kirchhoffa lub KCL, mówi, że algebraiczna suma prądów wpływających do dowolnego węzła jest równa zeru:

$$\sum_{k=1}^n i_k = 0. \quad (2.1)$$

Znak prądu zależy od przyjętej konwencji. Jeżeli prądy wpływające do węzła uznaje się za dodatnie, to prądy wypływające są ujemne. W praktyce oznacza to, że dla każdego węzła, który nie jest węzłem odniesienia, można zapisać jedno równanie bilansu prądów. Każdy element podłączony do danego węzła wnosi do tego równania składnik zależny od potencjałów węzłowych lub od wartości źródła.

Dla rezystora połączonego między węzłami a i b prąd można zapisać z prawa Ohma:

$$i_R = \frac{v_a - v_b}{R} = g(v_a - v_b), \quad (2.2)$$

gdzie R oznacza rezystancję, a $g = 1/R$ jest konduktancją. Po podstawieniu takiej zależności do równania KCL powstaje równanie liniowe względem napięć węzłowych. Właśnie ta własność powoduje, że rezystory i źródła prądowe są naturalnie opisywane metodami węzłowymi.

Drugie prawo Kirchhoffa, nazywane napięciowym prawem Kirchhoffa lub KVL, mówi, że algebraiczna suma napięć wzdłuż dowolnej zamkniętej pętli obwodu jest równa zeru:

$$\sum_{k=1}^m u_k = 0. \quad (2.3)$$

Prawo to opisuje zachowanie energii w zamkniętej drodze obwodu. Jeżeli przejście przez element odbywa się zgodnie z przyjętym kierunkiem spadku napięcia, składnik ma jeden znak, a przy przejściu przeciwnym znak przeciwny. W metodach oczkowych KVL jest zapisywane bezpośrednio jako równania dla oczek. W analizie węzłowej nie pojawia się zwykle jako osobny zestaw równań, lecz jest uwzględniane pośrednio przez definicję napięć jako różnic potencjałów między węzłami.

Szczególne znaczenie mają idealne źródła napięciowe. Źródło napięciowe połączone między węzłami a i b narzuca warunek:

$$v_a - v_b = V_s. \quad (2.4)$$

Tego równania nie da się w prosty sposób zapisać jako prądu zależnego wyłącznie od potencjałów węzłowych. Z tego powodu idealne źródła napięciowe i część źródeł sterowanych wymagają rozszerzenia klasycznej metody potencjałów węzłowych.

2.2 Metody macierzowe w analizie obwodów

Symulacja komputerowa wymaga sprowadzenia równań obwodu do postaci możliwej do rozwiązania numerycznego. Dla obwodów liniowych otrzymuje się układ równań liniowych:

$$Ax = b, \quad (2.5)$$

gdzie A jest macierzą współczynników, x jest wektorem niewiadomych, a b jest wektorem wymuszeń. W zależności od zastosowanej metody niewiadomymi mogą być napięcia węzłowe, prądy oczkowe albo dodatkowe prądy wprowadzone przez źródła napięciowe.

Podejście macierzowe jest szczególnie wygodne w programie komputerowym, ponieważ pozwala oddzielić etap tworzenia równań od etapu ich rozwiązywania. Elementy obwodu dodają swoje lokalne wkłady do globalnej macierzy i wektora wymuszeń, a następnie solver rozwiązuje gotowy układ. W przypadku większych układów ten podział ma także znaczenie organizacyjne, ponieważ każdy typ elementu może być opisany osobnym zestawem reguł.

Do rozwiązywania układu równanie (2.5) można wykorzystać standardowe metody algebry liniowej, na przykład eliminację Gaussa albo rozkład LU. W analizie czasowej lub w obecności elementów dynamicznych równania różniczkowe są najpierw dyskretyzowane, a następnie w każdym kroku czasu również sprowadzane do układu liniowego [5], [8].

2.3 Klasyczna metoda potencjałów węzłowych

Klasyczna metoda potencjałów węzłowych polega na wybraniu jednego węzła odniesienia, zwykle masy, oraz zapisaniu potencjałów pozostałych węzłów jako niewiadomych. Dla każdego węzła niebędącego masą zapisuje się równanie KCL. Prądy przez elementy są wyrażane za pomocą różnic potencjałów między węzłami.

Dla obwodu złożonego z elementów admitancyjnych i źródeł prądowych układ równań można zapisać w postaci:

$$Gv = i, \quad (2.6)$$

gdzie G jest macierzą konduktancji, v jest wektorem napięć węzłowych, a i jest wektorem prądów wstrzykiwanych do węzłów przez źródła prądowe.

Macierz G ma charakterystyczną strukturę. Elementy diagonalne są sumą konduktancji elementów dołączonych do danego węzła, natomiast elementy pozadiagonalne zawierają ujemne sumy konduktancji łączących pary węzłów. Dla rezystora między węzłami a i b do macierzy dodawane są cztery wkłady:

$$G_{aa} += g, \quad G_{bb} += g, \quad G_{ab} -= g, \quad G_{ba} -= g. \quad (2.7)$$

Jeżeli jeden z zacisków elementu jest połączony z masą, odpowiednie wpisy związane z węzłem odniesienia nie są dodawane jako osobne niewiadome. Dzięki temu masa pozostaje punktem odniesienia, a rozmiar układu jest mniejszy.

Ograniczenie klasycznej metody potencjałów węzłowych pojawia się przy elementach, które narzucają napięcie, ale nie definiują bezpośrednio prądu jako funkcji napięć węzłowych. Przykładem jest idealne źródło napięciowe. Można próbować rozwiązywać takie przypadki przez superwęzły albo dodatkowe przekształcenia, jednak w programie komputerowym wygodniejsze jest ujednolicenie zapisu przez wprowadzenie dodatkowych niewiadomych.

2.4 Zmodyfikowana metoda potencjałów węzłowych

Zmodyfikowana metoda potencjałów węzłowych, czyli Modified Nodal Analysis, rozszerza klasyczną metodę potencjałów węzłowych o dodatkowe niewiadome prądowe [4], [5]. Najczęściej są to prądy płynące przez idealne źródła napięciowe oraz przez elementy sterowane, które narzucają zależności napięciowe. Dzięki temu elementy niewygodne dla klasycznej analizy węzłowej mogą być opisane bez ręcznego tworzenia superwęzłów i bez zastępowania idealnych źródeł elementami pomocniczymi.

Ogólną postać układu równań ZMPW można zapisać blokowo:

$$\begin{bmatrix} G & B \\ C & D \end{bmatrix} \begin{bmatrix} v \\ j \end{bmatrix} = \begin{bmatrix} i \\ e \end{bmatrix}. \quad (2.8)$$

Wektor v zawiera napięcia węzłowe, natomiast j zawiera dodatkowe niewiadome prądowe. Macierz G opisuje zależności wynikające z elementów admitancyjnych, takich jak rezystory. Macierze B i C łączą równania węzłowe z dodatkowymi niewiadomymi, a macierz D opisuje zależności między dodatkowymi równaniami. Wektor i zawiera wymuszenia prądowe, a wektor e wymuszenia napięciowe.

Dla idealnego źródła napięciowego między węzłami a i b wprowadzana jest dodatkowa niewiadoma j_s , czyli prąd płynący przez źródło. Do układu dodawane jest równanie:

$$v_a - v_b = V_s. \quad (2.9)$$

Jednocześnie prąd j_s pojawia się w równaniach KCL dla węzłów a i b . W ten sposób źródło napięciowe może być opisane bez zastępowania go sztuczną rezystancją. Szczegółowy sposób zapisywania lokalnych wkładów elementów do macierzy ZMPW został opisany w następnym rozdziale.

2.5 Punkt pracy DC

Punkt pracy DC opisuje stan ustalony obwodu przy wymuszeniach stałych. Dla układów liniowych obliczenie punktu pracy sprowadza się do zbudowania macierzy ZMPW i rozwiązania jednego układu równań liniowych. Wynikiem są napięcia węzłowe oraz, jeśli zastosowano dodatkowe niewiadome, prądy przez wybrane elementy.

W analizie punktu pracy wielkości nie zależą od czasu, dlatego pochodne czasowe są równe zero. W idealnym modelu kondensator w stanie ustalonym zachowuje się jak przerwa, ponieważ prąd stały przez kondensator nie płynie. Cewka zachowuje się jak zwarcie, ponieważ napięcie na idealnej cewce w stanie ustalonym jest równe zero.

Dla obwodów nieliniowych punkt pracy wymaga zwykle metody iteracyjnej, na przykład metody Newtona-Raphsona. Wtedy element nieliniowy jest w każdej iteracji zastępowany lokalnym modelem liniowym, a układ równań jest rozwiązywany wielokrotnie do osiągnięcia zbieżności [2], [8]. W pracy punkt pracy stanowi podstawową analizę potrzebną do sprawdzenia poprawności zbudowanego układu równań oraz do wyznaczenia warunków początkowych dla analizy czasowej.

2.6 Analiza czasowa

Analiza czasowa opisuje odpowiedź układu w funkcji czasu. Jest potrzebna wtedy, gdy wymuszenia zmieniają się w czasie albo gdy w układzie występują elementy magazynujące energię,

takie jak kondensatory i cewki. W takim przypadku równania obwodu zawierają pochodne czasowe, które w symulatorach obwodowych są zastępowane równaniami dyskretnymi rozwiązywanymi w kolejnych chwilach czasu [2], [3].

Dla liniowego układu dynamicznego równania można zapisać w postaci:

$$C \frac{dx(t)}{dt} + Gx(t) = b(t), \quad (2.10)$$

gdzie C jest macierzą związaną z elementami dynamicznymi, G opisuje część rezystancyjną, $x(t)$ jest wektorem niewiadomych zależnych od czasu, a $b(t)$ jest wektorem wymuszeń.

Aby rozwiązać taki układ numerycznie, czas jest dzielony na kroki o długości Δt . Pochodne zastępuje się przybliżeniami dyskretnymi. Dla wstecznej metody Eulera pochodną można zapisać jako:

$$\frac{dx(t)}{dt} \approx \frac{x_k - x_{k-1}}{\Delta t}. \quad (2.11)$$

Po podstawieniu do równanie (2.10) otrzymuje się układ liniowy rozwiązywany w kroku k :

$$\left(G + \frac{C}{\Delta t} \right) x_k = b_k + \frac{C}{\Delta t} x_{k-1}. \quad (2.12)$$

Oznacza to, że analiza czasowa może być zrealizowana jako sekwencja obliczeń podobnych do punktu pracy, wykonywanych dla kolejnych chwil czasu. W każdym kroku aktualizowane są wkłady elementów dynamicznych oraz wymuszenia zależne od czasu. Stabilność i dokładność wyniku zależą od zastosowanej metody całkowania, wartości kroku czasowego oraz właściwego przygotowania warunków początkowych.

Alternatywnym podejściem jest metoda trapezów, która zwykle daje większą dokładność, ale w niektórych przypadkach może prowadzić do oscylacji numerycznych. Z tego powodu wybór metody całkowania jest kompromisem między dokładnością, stabilnością i kosztem obliczeniowym [2], [3].

Rozdział 3

Zmodyfikowana metoda potencjałów węzłowych i stemple elementów

3.1 Rola stempli w metodzie ZMPW

Zmodyfikowana metoda potencjałów węzłowych pozwala sprowadzić opis obwodu do jednego globalnego układu równań. W praktycznej implementacji wygodne jest jednak, aby nie tworzyć całej macierzy jednocześnie. Zamiast tego każdy element obwodu można opisać przez lokalny wkład do macierzy układu i wektora wymuszeń. Taki lokalny wkład nazywany jest stemplem elementu [4], [5].

Stempel określa, które wartości należy dodać do globalnej macierzy oraz do wektora prawej strony. Dzięki temu opis elementu jest oddzielony od całego obwodu. Rezystor, źródło prądowe, źródło napięciowe i źródła sterowane mogą być opisane osobnymi regułami, a globalny układ równań powstaje przez sumowanie ich wkładów.

Takie podejście jest zgodne z postacią ZMPW:

$$\begin{bmatrix} G & B \\ C & D \end{bmatrix} \begin{bmatrix} v \\ j \end{bmatrix} = \begin{bmatrix} i \\ e \end{bmatrix}. \quad (3.1)$$

Elementy admitancyjne, takie jak rezystory, modyfikują przede wszystkim blok G . Źródła prądowe dodają wartości do wektora i . Źródła napięciowe oraz wybrane źródła sterowane wprowadzają dodatkowe niewiadome prądowe, a więc rozszerzają macierz o nowe wiersze i kolumny.

3.2 Ogólna postać stempla

Stempel elementu można traktować jako małą macierz i mały wektor, którym przypisano konkretne indeksy w globalnym układzie równań. Dla elementu podłączonego do kilku węzłów lokalne indeksy odpowiadają numerom tych węzłów oraz ewentualnym dodatkowym niewia-

domym.

Ogólnie wkład elementu można zapisać jako:

$$A_{global}(r_p, c_q) += A_{local}(p, q), \quad (3.2)$$

gdzie A_{local} jest lokalną macierzą elementu, a r_p i c_q oznaczają odpowiadające jej indeksy w macierzy globalnej. Analogicznie wartości lokalnego wektora wymuszeń są dodawane do odpowiednich pozycji globalnego wektora prawej strony.

Jeżeli jeden z zacisków elementu jest połączony z masą, jego indeks nie tworzy osobnej niewiadomej. W takim przypadku część wpisów stempla jest pomijana albo dodawana wyłącznie do równań pozostałych węzłów. Dzięki temu potencjał masy pozostaje równy zeru i nie zwiększa rozmiaru układu równań.

3.3 Stempel rezystora

Rezystor jest podstawowym przykładem elementu, którego stempel wynika bezpośrednio z prawa Ohma i równania KCL. Dla rezystora o rezystancji R konduktancja wynosi:

$$g = \frac{1}{R}. \quad (3.3)$$

Jeżeli rezystor jest połączony między węzłami a i b , prąd płynący od węzła a do węzła b można zapisać jako:

$$i_R = g(v_a - v_b). \quad (3.4)$$

Po podstawieniu tej zależności do równań KCL dla obu węzłów otrzymuje się cztery wpisy do macierzy konduktancji:

$$G_{aa} += g, \quad G_{bb} += g, \quad G_{ab} -= g, \quad G_{ba} -= g. \quad (3.5)$$

Wpisy diagonalne zwiększają sumę konduktancji podłączonych do danego węzła, natomiast wpisy pozadiagonalne opisują sprzężenie między węzłami. Taki stempel jest symetryczny, co wynika z pasywnego charakteru rezystora.

3.4 Stemple źródeł niezależnych

Niezależne źródło prądowe nie wprowadza nowej niewiadomej. Jego wpływ polega na dodaniu wartości prądu do wektora wymuszeń. Dla źródła prądowego podłączonego między węzłami a i b , przy przyjętym kierunku prądu od a do b , wkład można zapisać jako:

$$i_a -= I_s, \quad i_b += I_s. \quad (3.6)$$

Znaki zależą od przyjętej konwencji dla prądów wpływających i wypływających z węzła. Najważniejsze jest zachowanie spójności tej konwencji w całej macierzy.

Niezależne źródło napięciowe wymaga rozszerzenia układu równań. Dla źródła między węzłami a i b wprowadza się dodatkową niewiadomą j_s , oznaczającą prąd przez źródło. Równanie napięciowe ma postać:

$$v_a - v_b = V_s. \quad (3.7)$$

W macierzy pojawiają się wpisy łączące nową niewiadomą z równaniami KCL węzłów a i b , a także nowy wiersz opisujący narzucone napięcie:

$$A_{a,s} += 1, \quad A_{b,s} -= 1, \quad A_{s,a} += 1, \quad A_{s,b} -= 1, \quad b_s += V_s. \quad (3.8)$$

Indeks s oznacza dodatkowy wiersz i kolumnę związane z prądem źródła. Ten mechanizm jest jednym z najważniejszych powodów stosowania ZMPW.

3.5 Stemple źródeł sterowanych

Źródła sterowane opisują zależność jednej wielkości elektrycznej od innej wielkości w obwodzie. W metodzie ZMPW można je wprowadzać w sposób zbliżony do źródeł niezależnych, ale z dodatkowymi współczynnikami zależnymi od sygnału sterującego.

Źródło prądowe sterowane napięciem, czyli VCCS, nie wymaga dodatkowej niewiadomej. Jeżeli prąd wyjściowy zależy od napięcia sterującego $v_c - v_d$, to:

$$i = g_m(v_c - v_d), \quad (3.9)$$

gdzie g_m oznacza transkonduktancję. Wkład takiego źródła trafia do równań KCL węzłów wyjściowych, a współczynniki przy napięciach sterujących są dodawane do odpowiednich kolumn macierzy.

Źródło napięciowe sterowane napięciem, czyli VCVS, narzuca równanie napięciowe:

$$v_a - v_b = \mu(v_c - v_d), \quad (3.10)$$

gdzie μ jest wzmocnieniem napięciowym. Ponieważ źródło narzuca napięcie między węzłami wyjściowymi, wymaga dodatkowej niewiadomej prądowej podobnie jak niezależne źródło napięciowe. Różnica polega na tym, że w dodatkowym równaniu pojawiają się również współczynniki przy węzłach sterujących.

Źródło prądowe sterowane prądem, czyli CCCS, wymaga informacji o prądzie sterującym. Jeżeli prąd sterujący jest jedną z dodatkowych niewiadomych ZMPW, źródło może dodać odpowiednie współczynniki do równań KCL węzłów wyjściowych. Źródło napięciowe sterowane prądem, czyli C CVS, dodatkowo narzuca napięcie wyjściowe zależne od prądu sterującego:

$$v_a - v_b = r_m j_c, \quad (3.11)$$

gdzie r_m oznacza transrezystancję, a j_c prąd sterujący. Taki element również wprowadza dodatkowy wiersz związany z napięciem wyjściowym.

3.6 Stemple elementów nieliniowych

Elementy liniowe mają stemple niezależne od aktualnego punktu pracy. Rezystor o stałej rezystancji zawsze wnosi ten sam zestaw współczynników do macierzy, a niezależne źródło napięciowe zawsze narzuca to samo równanie napięciowe. Inaczej zachowują się elementy nieliniowe, których prąd nie jest liniową funkcją napięć węzłowych. Przykładem takiego elementu jest tranzystor bipolarny BJT.

Dla elementu nieliniowego nie wystarcza jeden stały stempel. W każdej iteracji obliczeń element jest linearyzowany wokół aktualnego przybliżenia rozwiązania. Oznacza to, że jego wkład do macierzy zależy od bieżących napięć węzłowych. Solver musi więc przekazać elementowi kontekst obliczeń, a element na tej podstawie zwraca lokalny model liniowy obowiązujący tylko dla danej iteracji.

W programie ten mechanizm jest reprezentowany przez wspólny kontekst stempla. Zawiera on między innymi aktualne przybliżenie wektora rozwiązania, czas symulacji, krok czasowy oraz rozwiązanie z poprzedniego kroku. Dzięki temu ten sam interfejs może obsługiwać zarówno źródło sinusoidalne zależne od czasu, kondensator zależny od poprzedniego stanu, jak i tranzystor BJT zależny od bieżącego punktu pracy.

Linearyzacja elementów nieliniowych jest połączona z metodą Newtona-Raphsona. W każdej iteracji tworzony jest zlinearyzowany układ równań, następnie wyznaczane jest nowe przybliżenie rozwiązania, a proces jest powtarzany do osiągnięcia zbieżności. W przypadku BJT oznacza to, że przewodności małosygnałowe i prądy zastępcze są obliczane ponownie dla kolejnych przybliżeń napięć bazy, kolektora i emitera.

3.7 Wypełnianie macierzy globalnej

Globalna macierz układu powstaje przez zsumowanie wkładów wszystkich elementów. Każdy stempel modyfikuje tylko wybrane pozycje macierzy i wektora prawej strony. Pozostałe pozycje pozostają niezmienione. Proces ten można przedstawić jako dodawanie lokalnych równań do jednej wspólnej struktury macierzowej.

Najpierw ustalany jest rozmiar układu równań. Zależy on od liczby węzłów niebędących masą oraz od liczby dodatkowych niewiadomych wymaganych przez źródła napięciowe i wybrane źródła sterowane. Następnie dla każdego elementu dodawane są wpisy wynikające z jego stempla.

Taki sposób budowania macierzy jest modularny. Dodanie nowego typu elementu nie wymaga zmiany całej metody rozwiązywania układu. Wystarczy zdefiniować jego lokalny wkład do macierzy i wektora wymuszeń. Jest to jedna z głównych zalet opisu opartego na stemplach, wykorzystywana również w klasycznych symulatorach obwodowych [2], [3].

3.8 Znaczenie dodatkowych wierszy

Dodatkowe wiersze i kolumny pojawiają się wtedy, gdy sam opis napięć węzłowych nie wystarcza do przedstawienia zachowania elementu. Typowym przykładem jest idealne źródło napięciowe. Prąd przez takie źródło nie wynika bezpośrednio z napięcia na jego zaciskach, dlatego musi zostać dodany jako nowa niewiadoma.

Każda dodatkowa niewiadoma zwiększa rozmiar układu równań o jeden. Jednocześnie pojawia się nowe równanie opisujące zależność narzuconą przez element. W dobrze zbudowanym układzie liczba niewiadomych i liczba równań pozostają sobie równe.

W praktyce poprawne zarządzanie dodatkowymi wierszami ma duże znaczenie dla stabilności całego algorytmu. Błędne przypisanie indeksu dodatkowej niewiadomej może prowadzić do nadpisywania równań, pustych wierszy lub macierzy osobliwej. Dlatego elementy wymagające dodatkowych niewiadomych muszą być opisywane konsekwentnie i z zachowaniem jednoznacznej numeracji.

Rozdział 4

Założenia i architektura aplikacji

4.1 Założenia projektowe

Celem części programistycznej pracy jest przygotowanie aplikacji, która pozwala przejść od prostego opisu tekstowego obwodu do wyniku analizy numerycznej. Program nie jest traktowany jako uniwersalny zamiennik rozbudowanych środowisk SPICE, lecz jako dedykowane narzędzie pokazujące, w jaki sposób metoda ZMPW może zostać zaimplementowana w natywnej aplikacji napisanej w języku Swift.

Najważniejsze założenie projektowe polega na oddzieleniu opisu obwodu od sposobu jego rozwiązania. Użytkownik może wpisać układ bezpośrednio w głównym oknie programu albo przeciągnąć do niego plik tekstowy z netlistą. Aplikacja przekształca ten opis na obiekty reprezentujące stemple elementów. Dopiero tak przygotowana lista stempli jest przekazywana do solvera, który buduje macierz globalną i wyznacza wynik wybranej analizy.

Takie podejście pozwala rozwijać program stopniowo. Dodanie nowego elementu wymaga zdefiniowania jego stempla oraz rozszerzenia warstwy parsera o regułę rozpoznawania nowego oznaczenia. Sam solver nie musi znać nazw elementów ani formatu pliku wejściowego, ponieważ operuje wyłącznie na macierzach zwracanych przez stemple.

4.2 Ogólna architektura

Program został podzielony na kilka warstw odpowiedzialności. Pierwszą z nich jest interfejs użytkownika, który udostępnia edytor opisu obwodu, obsługę przeciągania plików .txt oraz przycisk uruchamiający symulację. Drugą warstwą jest parser tekstu wejściowego. Jego zadaniem jest odczytanie kolejnych linii, rozpoznanie typu elementu, sprawdzenie liczby parametrów oraz utworzenie listy obiektów typu Stamp. Trzecią warstwę tworzą klasy elementów obwodu, na przykład rezystor, źródła niezależne i źródła sterowane. Każdy z tych elementów zwraca lokalny wkład do macierzy przewodności oraz do wektora wymuszeń. Czwartą warstwą jest solver, który sumuje wszystkie stemple i rozwiązuje powstały układ równań.

Na poziomie przepływu danych aplikacja wykonuje następujące kroki:

1. pobiera tekstowy opis obwodu z edytora albo z przeciągniętego pliku,
2. dzieli opis na linie i tokeny,
3. rozpoznaje typy elementów oraz komendę analizy,
4. tworzy obiekty stempli i przypisuje dodatkowe indeksy równań,
5. przekazuje listę stempli oraz komendę do solvera,
6. buduje macierz globalną i oblicza wynik analizy,
7. prezentuje wynik punktu pracy albo wykres przebiegów czasowych.

Wynikiem pracy parsera jest struktura zawierająca tablicę stempli, komendę solvera oraz listę węzłów, których przebiegi mają zostać pokazane na wykresie. Dzięki temu parser nie rozpoczyna obliczeń samodzielnie. Jego odpowiedzialność kończy się na przygotowaniu danych wejściowych w formie zrozumiałej dla dalszych części programu.

Cały przepływ danych można przedstawić jako przejście od interfejsu użytkownika albo pliku tekstowego do struktury `ParserResult`, a następnie do solvera i modułu wizualizacji. Dla komendy `.op` solver zwraca pojedynczy wektor rozwiązania. Dla komendy `.tran` zwraca sekwencję próbek czasowych, które moduł `Plotter` zamienia na serie punktów dla węzłów wskazanych przez `.show`. Interfejs użytkownika nie interpretuje samodzielnie składni netlisty, lecz przekazuje tekst do parsera i wybiera odpowiedni widok wyniku na podstawie zwróconej komendy.

Rysunek 4.1 pokazuje główne okno aplikacji. Centralną część zajmuje edytor netlisty, a po prawej stronie znajduje się obszar do przeciągania plików tekstowych. Po uruchomieniu symulacji program wybiera sposób prezentacji wyniku na podstawie komendy analizy: dla `.op` pokazuje wartości niewiadomych, a dla `.tran` rysuje wykres napięć wskazanych komendą `.show`.

BBSPICE

Circuit description

▶ Run Simulation

```
R 1 0 1000  
DCVS 1 0 5  
.op
```



Drop .txt file

File content will replace the
circuit description.

Rysunek 4.1: Główne okno aplikacji z edytorem opisu układu

4.3 Format pliku wejściowego

Opis obwodu ma postać tekstową zbliżoną do uproszczonej netlisty SPICE. Jedna linia opisu-je jeden element albo jedną komendę analizy. W przypadku elementu pierwszy token określa jego typ, kolejne tokeny oznaczają numery węzłów, a ostatnie wartości są parametrami elementu. Węzeł o numerze 0 pełni rolę węzła odniesienia. Taki format jest prosty do edycji ręcznej i dobrze pasuje do programowego parsowania opisu obwodu [2], [3].

Aktualnie obsługiwane są: rezystor R, kondensator C, niezależne źródło prądowe DCCS, niezależne źródła napięciowe DCVS i ACVS, źródła sterowane VCCS, VCVS, CCCS, C CVS oraz tranzystor bipolarny BJT. Komenda `.op` oznacza analizę punktu pracy, natomiast `.tran` uruchamia analizę czasową. W przypadku analizy czasowej można dodatkowo użyć komendy `.show`, która wskazuje węzły przeznaczone do wizualizacji.

Tabela 4.1: Format opisu obsługiwanych elementów

Oznaczenie	Postać linii	Znaczenie parametrów
R	R n1 n2 R	rezystor między węzłami n1 i n2
C	C n1 n2 C	kondensator między węzłami n1 i n2
DCCS	DCCS n+ n- I	niezależne źródło prądowe DC
DCVS	DCVS n+ n- V	niezależne źródło napięciowe DC
ACVS	ACVS n+ n- A f	sinusoidalne źródło napięciowe o amplitudzie A i częstotliwości f
VCCS	VCCS in+ in- out+ out- gm	źródło prądowe sterowane napięciem
VCVS	VCVS in+ in- out+ out- A	źródło napięciowe sterowane napięciem
CCCS	CCCS in+ in- out+ out- beta Vx	źródło prądowe sterowane prądem pomocniczego źródła napięciowego
CCVS	CCVS in+ in- out+ out- Rm Vx	źródło napięciowe sterowane prądem pomocniczego źródła napięciowego
BJT	BJT b c e Is beta m	tranzystor bipolarny opisany węzłami bazy, kolektora i emitera

Listing 4.1 przedstawia przykładową netlistę zawierającą wszystkie obsługiwane obecnie typy elementów. Sam układ nie musi mieć znaczenia praktycznego; jego zadaniem jest pokazanie składni wejściowej i kolejności parametrów.

```
R 1 0 1000
DCCS 0 1 0.001
DCVS 2 0 5
VCCS 1 0 3 0 0.5
VCVS 1 0 4 0 2
CCCS 2 0 5 0 3 1
CCVS 3 0 6 0 4 1
C 1 2 0.0000001
ACVS 7 0 2 1000
BJT 2 3 4 0.000000000001 100 1
.op
```

Listing 4.1: Przykładowy opis obwodu z wszystkimi obsługiwanyimi elementami

Dla źródeł napięciowych przyjęto konwencję zgodną z równaniami ZMPW: zapis DCVS n+ n- V oznacza warunek $V_{n+} - V_{n-} = V$. Analogicznie ACVS n+ n- A f oznacza wymuszenie sinusoidalne $V_{n+} - V_{n-} = A \sin(2\pi ft)$. Konwencja ta jest istotna, ponieważ od kolejności węzłów zależy znak wymuszenia.

Listing 4.2 pokazuje netlistę układu wtórnika emiterowego użytego do sprawdzenia analizy czasowej. Węzeł 1 jest wyjściem źródła sinusoidalnego, węzeł 2 odpowiada bazie tranzystora,

węzeł 3 kolektorowi, a węzeł 4 emiterowi. Komenda `.show 2 4` oznacza, że na wykresie mają zostać pokazane napięcia bazy i emitera.

```
R 3 2 27000
R 0 4 1000
ACVS 1 0 2 1000
DCVS 3 0 5
C 1 2 0.0000001
BJT 2 3 4 0.000000000001 100 1
.tran 0.004 0.00001
.show 2 4
```

Listing 4.2: Opis układu wtórnika emiterowego

4.4 Parser netlisty

Parser odczytuje plik linia po linii. Każda linia jest dzielona na tokeny rozdzielone białymi znakami. Linie puste oraz komentarze rozpoczynające się od znaku `*` są pomijane. Jeżeli pierwszy token jest komendą analizy, parser zapisuje ją jako polecenie dla solvera. W przeciwnym przypadku token jest interpretowany jako oznaczenie elementu.

Rozpoznawanie elementów jest wydzielone do typu `ElementType`. Typ ten pełni rolę słownika wspieranych elementów: zamienia tekstowe oznaczenie z pliku na konkretny wariant programu, określa wymaganą liczbę tokenów oraz wskazuje, które tokeny są numerami węzłów. Dzięki temu podstawowe reguły składniowe są skupione w jednym miejscu.

Parser wykonuje dwa przejścia po danych wejściowych. W pierwszym przejściu sprawdza strukturę linii, zapamiętuje poprawnie rozpoznane elementy i wyznacza największy numer węzła występujący w pliku. Drugie przejście służy do tworzenia stempli. Podział na dwa etapy jest potrzebny, ponieważ elementy zawierające dodatkowe równania, na przykład źródła napięciowe, wymagają indeksów większych od numerów zwykłych węzłów.

```
let parserResult = try Parser().parse(fileURL)
let stamps = parserResult.stamps
let command = parserResult.command
let result = try Solver().solve(stamps, command)
```

Listing 4.3: Uproszczony schemat odpowiedzialności parsera

4.5 Przypisywanie dodatkowych równań

W metodzie ZMPW część elementów wymaga wprowadzenia dodatkowej niewiadomej. Dotyczy to przede wszystkim źródeł napięciowych oraz źródeł sterowanych prądem. W imple-

mentacji taka niewiadoma jest reprezentowana przez dodatkowy indeks macierzy, nazywany w programie `newRow`.

Parser najpierw zlicza węzły układu, a następnie nadaje kolejne indeksy elementom wymagającym dodatkowych równań. Jeżeli największy numer węzła w pliku wynosi `nodeCount`, pierwszy dodatkowy indeks ma wartość `nodeCount + 1`, drugi `nodeCount + 2` i tak dalej. Źródło CCVS potrzebuje dwóch takich indeksów, ponieważ oprócz równania wyjściowego źródła napięciowego wymaga również równania umożliwiającego reprezentację prądu sterującego.

Ten mechanizm jest istotny, ponieważ użytkownik nie powinien ręcznie numerować równań pomocniczych. Numery w pliku wejściowym opisują wyłącznie węzły obwodu, natomiast wewnętrzna rozbudowa macierzy jest zadaniem programu.

4.6 Reprezentacja elementów i stempli

Każdy element obwodu jest reprezentowany przez klasę albo strukturę zwracającą stempel. Stempel składa się z lokalnej macierzy przewodności oraz lokalnego wektora wymuszeń. Dla rezystora niezerowe wartości trafiają wyłącznie do macierzy przewodności. Dla niezależnego źródła prądowego wkład pojawia się w wektorze wymuszeń. Źródła napięciowe oraz część źródeł sterowanych wprowadzają dodatkowe wiersze i kolumny w macierzy globalnej.

Istotne jest to, że obiekty elementów nie znają całej `netlisty`. Otrzymują jedynie numery węzłów, parametry elementu oraz ewentualne indeksy dodatkowych równań. Następnie zwracają macierze, które można dodać do macierzy globalnej. Dzięki temu kod odpowiedzialny za konkretny element pozostaje lokalny i łatwy do testowania.

4.7 Solver i komendy analizy

Solver otrzymuje listę stempli oraz komendę analizy. Dla analizy punktu pracy `.op` tworzy macierz globalną, dodaje do niej wkład każdego stempla i rozwiązuje układ równań liniowych. Wynikiem jest macierz wartości niewiadomych, czyli napięć węzłowych oraz prądów związanych z dodatkowymi równaniami.

Komenda `.tran` wykorzystuje tę samą ideę składania stempli, ale obliczenia są wykonywane w kolejnych chwilach czasu. Elementy dynamiczne są wtedy zastępowane równoważnymi modelami dyskretnymi zależnymi od kroku czasowego i historii poprzednich stanów. Dzięki wspólnej reprezentacji przez stemple analiza czasowa może korzystać z tej samej warstwy macierzy co punkt pracy [2], [5]. Wynik analizy czasowej może zostać przekazany do modułu wizualizacji, który tworzy serie punktów dla wskazanych węzłów.

Rysunek 4.2 pokazuje wynik analizy punktu pracy dla prostego dzielnika napięcia. Opis układu zawiera dwa rezystory $1\text{ k}\Omega$ oraz źródło napięciowe 5 V . Ponieważ rezystory mają takie same wartości, napięcie w węźle pośrednim wynosi połowę napięcia zasilania, czyli około $2,5\text{ V}$. Wynik programu pokazuje również 5 V w węźle połączonym z dodatnim zaciskiem źródła.

```
R 1 2 1000
R 2 0 1000
DCVS 1 0 5
.op
```

Listing 4.4: Netlista dzielnika napięcia użytego do analizy punktu pracy

BBSPICE

Operation point result

< Back

x1
5.000000

x2
2.500000

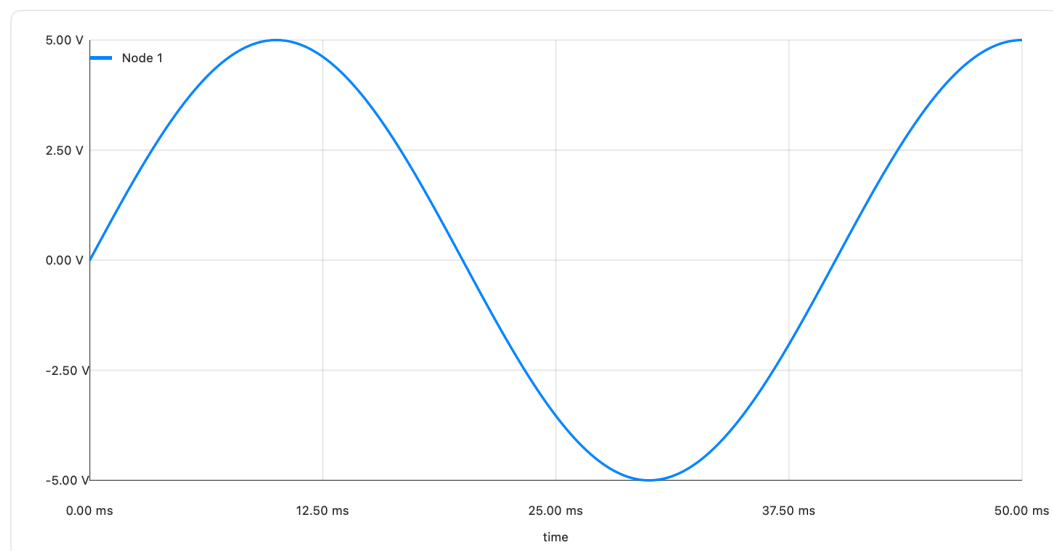
x3
-0.002500

Rysunek 4.2: Wynik analizy punktu pracy dla dzielnika napięcia

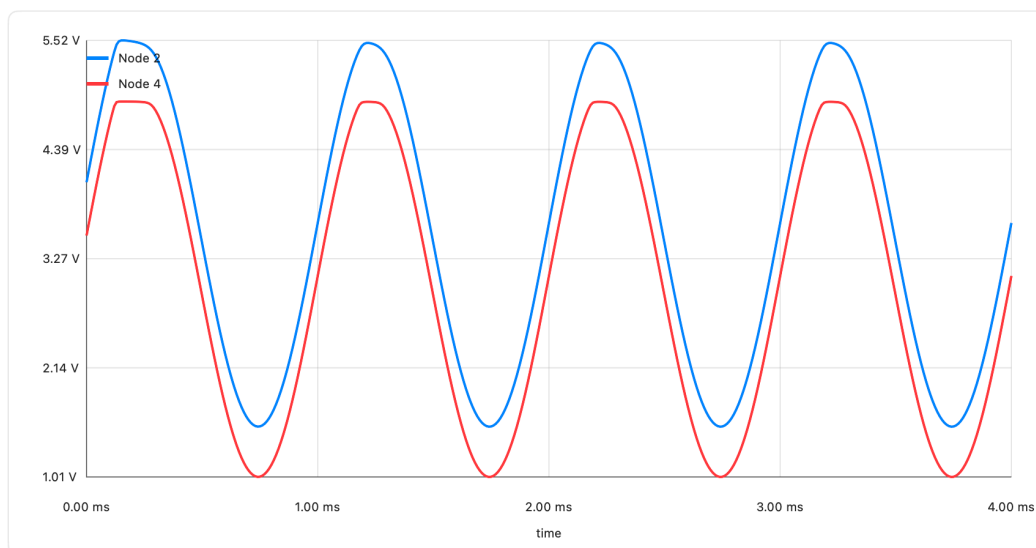
BBSPICE

Transient voltage result

< Back



Rysunek 4.3: Wynik analizy czasowej prostego układu ze źródłem sinusoidalnym i rezystorem



Rysunek 4.4: Wynik symulacji czasowej wtórnika emiterowego

Na wykresie wtórnika emiterowego widoczne są przebiegi napięcia bazy i emitera. Napięcie emitera podąża za napięciem bazy, ale jest przesunięte o spadek napięcia baza-emiter tranzystora. Taki wynik odpowiada oczekiwanej pracy wtórnika napięciowego: układ nie odwraca fazy sygnału i przenosi jego kształt na wyjście, ale poziom napięcia wyjściowego jest ograniczony przez właściwości złącza tranzystora. Przykład ten jest ważniejszy niż prosty układ liniowy, ponieważ jednocześnie wykorzystuje źródło sinusoidalne, kondensator, analizę czasową oraz nieliniowy model BJT.

Rozdzielenie parsera i solvera upraszcza dalszy rozwój aplikacji. Parser odpowiada za przekształcanie tekstu na dane programu, natomiast solver odpowiada za interpretację komendy analizy. W przyszłości ta sama struktura może zostać rozszerzona o kolejne polecenia, na przykład analizę małosygnałową albo warianty analizy czasowej z dodatkowymi parametrami kroku i czasu końcowego.

4.8 Obsługa błędów

Obsługa błędów została podzielona na błędy parsera i błędy stempli. Parser wykrywa problemy związane ze składnią pliku: nieznaną element, złą liczbę parametrów, niepoprawny typ parametru, brak komendy analizy albo podanie więcej niż jednej komendy. Każdy taki błąd zawiera numer linii, w której wystąpił problem, co ułatwia poprawienie pliku wejściowego.

Błędy stempli dotyczą wartości, które formalnie da się odczytać z pliku, ale które nie mają sensu z punktu widzenia elementu. Przykładem jest ujemny indeks węzła albo niepoprawna wartość parametru. Parser przechwytuje takie błędy i zamienia je na błąd związany z konkretną

linią netlisty. Użytkownik nie musi znać wewnętrznej struktury klas elementów, aby dowiedzieć się, gdzie w pliku znajduje się problem.

Takie rozdzielenie odpowiedzialności ułatwia utrzymanie programu. Parser nie musi znać szczegółowych warunków poprawności każdego elementu, a element nie musi wiedzieć, z której linii pliku pochodzi. Wspólnym punktem styku jest mechanizm wyjątków, który przenosi informację o niepowodzeniu do warstwy odpowiedzialnej za komunikat diagnostyczny.

Rozdział 5

Implementacja

5.1 Struktura kodu

Implementacja aplikacji została podzielona zgodnie z podziałem odpowiedzialności opisanym w rozdział 4. Kod źródłowy obejmuje moduły związane z interfejsem użytkownika, parserem, stemplami elementów, macierzami oraz solverem. Taki układ odpowiada naturalnemu przepływowi danych: od tekstowego opisu obwodu, przez reprezentację elementów, aż do układu równań rozwiązywanego numerycznie i prezentacji wyniku. Język Swift został wykorzystany ze względu na typowanie statyczne, obsługę błędów i dobre wsparcie dla aplikacji macOS [6].

Warstwa widoku odpowiada za przyjmowanie netlisty od użytkownika i wybór formy prezentacji wyniku. Warstwa parsera zawiera typy odpowiedzialne za rozpoznawanie elementów i komend analizy. Warstwa stempli zawiera klasy opisujące konkretne elementy obwodu. Warstwa matematyczna udostępnia macierze oraz operacje potrzebne do ich składania i rozwiązywania. Solver łączy część obliczeniową, ponieważ otrzymuje gotową listę stempli i wykonuje analizę wskazaną przez komendę.

W implementacji przyjęto założenie, że elementy nie powinny samodzielnie interpretować tekstu wejściowego. Każdy konstruktor elementu otrzymuje już przetworzone wartości liczbowe: numery węzłów, parametry oraz ewentualne indeksy dodatkowych równań. Dzięki temu błędy składniowe pozostają w parserze, a błędy fizycznie lub numerycznie niepoprawnych parametrów pozostają w warstwie stempli.

5.2 Parser i wynik parsowania

Parser pracuje na tekście zakodowanym jako UTF-8. Tekst może pochodzić z pliku albo bezpośrednio z pola edycji w interfejsie użytkownika. Każda linia jest dzielona na tokeny, a pierwszy token decyduje o dalszej interpretacji. Jeżeli token odpowiada komendzie analizy, parser zapisuje ją jako polecenie dla solvera. Jeżeli token odpowiada elementowi, parser sprawdza liczbę parametrów oraz zapamiętuje linię do późniejszego utworzenia stempla.

Rezultatem parsowania jest struktura zawierająca listę stempli, komendę analizy oraz listę węzłów przeznaczonych do wizualizacji. Taki wynik jest wygodny, ponieważ stanowi pełne wejście dla solvera i widoku wyniku, ale nie wiąże parsera z konkretną metodą obliczeń.

```
struct ParserResult {  
    let stamps: [Stamp]  
    let command: SolverCommand  
    let showNodes: [Int]  
}
```

Listing 5.1: Model wyniku parsowania

Ważnym elementem parsera jest typ `ElementType`. Odpowiada on za powiązanie oznaczeń tekstowych z obsługiwanymi elementami programu. Ten typ przechowuje również informacje o wymaganej liczbie tokenów oraz o pozycjach tokenów, które należy traktować jako numery węzłów.

```
enum ElementType {  
    case resistor  
    case capacitor  
    case DCCS  
    case DCVS  
    case ACVS  
    case CCCS  
    case VCCS  
    case C CVS  
    case V CVS  
    case BJT  
}
```

Listing 5.2: Uproszczona rola typu `ElementType`

Takie rozwiązanie ogranicza liczbę miejsc, w których trzeba zmieniać kod po dodaniu kolejnego elementu. Nowy element wymaga rozszerzenia słownika typów, określenia liczby parametrów oraz dodania funkcji tworzącej właściwy stempel.

5.3 Tworzenie stempli

Podstawowym typem wspólnym dla elementów jest `Stamp`. Udostępnia on metody zwracające lokalną macierz przewodności oraz lokalny wektor wymuszeń. Klasy konkretnych elementów nadpisują te metody zgodnie z równaniami wynikającymi z metody ZMPW.

```
class Stamp {  
    func getGMatrix(_ context: StampContext) throws -> Matrix? {
```

```

        return nil
    }

    func getIMatrix(_ context: StampContext) throws -> Matrix? {
        return nil
    }
}

```

Listing 5.3: Wspólny interfejs stempla

Argument `StampContext` przekazuje do elementu informacje potrzebne w danym rodzaju analizy. Dla analizy czasowej zawiera między innymi aktualny czas, krok czasowy, bieżące przybliżenie rozwiązania oraz rozwiązanie z poprzedniego kroku. Dzięki temu ten sam interfejs stempla może obsługiwać elementy liniowe, elementy dynamiczne i elementy nieliniowe.

Wprowadzenie kontekstu upraszcza rozwój programu, ponieważ element nie musi posiadać wielu wersji metod dla różnych analiz. Źródło ACVS odczytuje z kontekstu aktualny czas i na tej podstawie oblicza chwilową wartość sinusoidy. Kondensator wykorzystuje krok czasowy oraz poprzednie napięcie, aby utworzyć model zastępczy dla analizy `.tran`. Tranzystor BJT korzysta z bieżącego przybliżenia rozwiązania, ponieważ jego zlinearyzowany stempel zależy od aktualnych napięć bazy, kolektora i emitera.

Dla elementów pasywnych, takich jak rezystor, najważniejsza jest macierz G . Dla niezależnego źródła prądowego zasadniczy wkład trafia do wektora I . Źródła napięciowe i wybrane źródła sterowane wymagają dodatkowego równania, dlatego ich stemple są większe i obejmują dodatkowe wiersze oraz kolumny.

Z punktu widzenia implementacji istotne jest to, że stempel nie musi znać pełnego rozmiaru układu. Zwraca macierz o takim rozmiarze, który jest potrzebny do zapisania jego największego indeksu. Dopiero solver tworzy macierz globalną o rozmiarze dopasowanym do największego stempla w całej liście.

5.4 Macierze

Macierz jest reprezentowana przez liczbę wierszy, liczbę kolumn oraz jednowymiarową tablicę wartości typu `Double`. Dostęp do elementu macierzy odbywa się przez przeliczenie indeksu wiersza i kolumny na indeks tablicy. W implementacji numery węzłów i równań są zgodne z notacją obwodową, dlatego indeks 0 oznacza masę i nie jest zapisywany w macierzy.

Dodawanie wartości do macierzy ma charakter akumulacyjny. Jeżeli dwa elementy wnoszą wkład do tej samej pozycji, wartości są sumowane. Jest to bezpośrednie odzwierciedlenie składania równań ZMPW, gdzie każdy element dopisuje lokalny fragment do globalnego układu.

```

func add(_ row: Int, _ column: Int, _ value: Double) throws {
    if row == 0 || column == 0 || value == 0 { return }
}

```

```

        values[columns * (row - 1) + (column - 1)] += value
    }

```

Listing 5.4: Zasada dodawania wartości do macierzy

W programie wyróżniono klasy pomocnicze GMatrix, IMatrix oraz VMatrix. Pierwsza reprezentuje macierz układu, druga wektor prawej strony, a trzecia wektor rozwiązania. Ten podział nie zmienia sposobu przechowywania danych, ale zwiększa czytelność kodu.

5.5 Solver

Solver rozpoczyna pracę od wyznaczenia wymaganego rozmiaru macierzy globalnej. W tym celu przegląda wszystkie stemple i sprawdza rozmiary zwracanych przez nie macierzy. Następnie tworzy pustą macierz G oraz pusty wektor I, po czym dodaje do nich wkład każdego elementu.

```

for stamp in stamps {
    try gMatrix.add(stamp.getGMatrix(context))
    try iMatrix.add(stamp.getIMatrix(context))
}

```

Listing 5.5: Schemat składania macierzy globalnej

Dla analizy punktu pracy układ ma postać

$$\mathbf{G}\mathbf{x} = \mathbf{I}. \quad (5.1)$$

Po złożeniu macierzy solver rozwiązuje ten układ i zwraca wektor niewiadomych. Wektor zawiera napięcia węzłowe oraz dodatkowe niewiadome wprowadzone przez źródła napięciowe i źródła sterowane prądem.

Analiza czasowa korzysta z tej samej struktury organizacyjnej. Różnica polega na tym, że macierze dla elementów dynamicznych zależą od kroku czasowego i poprzednich stanów układu. Przed rozpoczęciem analizy .tran solver wyznacza punkt pracy, a następnie traktuje go jako warunek początkowy dla pierwszego kroku czasowego.

W kolejnych krokach czasowych solver tworzy nowy kontekst zawierający aktualny czas, krok czasowy i rozwiązanie z poprzedniego kroku. Następnie składa macierz globalną z tych samych stempli, ale ich wartości mogą być inne niż w poprzednim kroku. Dla źródła sinusoidalnego zmienia się wartość wymuszenia, dla kondensatora zmienia się wkład zależny od historii, a dla elementów nieliniowych zmienia się lokalna linearyzacja.

W przypadku elementów nieliniowych, takich jak tranzystor bipolarny, solver używa iteracji Newtona-Raphsona. W każdej iteracji stemple są obliczane dla aktualnego przybliżenia rozwiązania, a następnie rozwiązywany jest zlinearyzowany układ równań. Proces kończy się po osiągnięciu zadanej tolerancji albo po przekroczeniu maksymalnej liczby iteracji. W aktual-

nej implementacji limit ustawiono na 150 iteracji, co pozwala stabilnie policzyć układ wtórnika emiterowego wykorzystany jako przykład analizy czasowej.

5.6 Interfejs użytkownika i wizualizacja wyników

Interfejs użytkownika został zaimplementowany w SwiftUI. Główny widok ma trzy podstawowe stany: edycję opisu układu, prezentację wyniku punktu pracy oraz prezentację wyniku analizy czasowej. W stanie edycji użytkownik może wpisać netlistę ręcznie albo przeciągnąć plik `.txt`; po upuszczeniu pliku jego zawartość zastępuje tekst w edytorze. Do rozpoznawania przeciąganego typu pliku wykorzystano mechanizm Uniform Type Identifiers dostępny w systemowych bibliotekach Apple [9], [10].

Po naciśnięciu przycisku uruchomienia program przekazuje tekst do parsera. Jeżeli komendą analizy jest `.op`, widok pokazuje wartości kolejnych niewiadomych wektora rozwiązania. Jeżeli komendą jest `.tran`, solver zwraca kolejne próbki czasowe, a widok przechodzi do wykresu. W obu przypadkach użytkownik może wrócić do edytora bez utraty wpisanego opisu układu.

Do prezentacji wyników analizy czasowej dodano moduł `Plotter`. Nie wykonuje on obliczeń obwodowych, lecz przekształca wynik solvera na serie punktów opisane czasem i napięciem wybranego węzła. Lista węzłów pochodzi z komendy `.show`, dzięki czemu opis wejściowy decyduje o tym, które przebiegi zostaną pokazane.

Warstwa widoku korzysta z przygotowanych serii i rysuje wykres napięcia w funkcji czasu. Takie rozdzielenie ułatwia dalszy rozwój programu: solver pozostaje odpowiedzialny wyłącznie za obliczenia, a wizualizacja może zostać rozbudowana niezależnie, na przykład o eksport danych albo porównywanie kilku symulacji.

5.7 Obsługa błędów w implementacji

Błędy zostały podzielone na kilka grup. `ParserError` opisuje problemy z plikiem wejściowym, takie jak nieznaną element, złą liczbę parametrów, niepoprawny typ parametru, brak komendy, więcej niż jedna komenda analizy, więcej niż jedna komenda `.show` albo użycie `.show` bez analizy czasowej. Każdy błąd parsera związany z konkretną linią zawiera jej numer.

`StampParameterError` opisuje problemy wykryte przez elementy. Przykładem jest ujemny indeks węzła albo wartość parametru, której nie można użyć w obliczeniach. Parser przechwytywa takie błędy i zamienia je na informację diagnostyczną odnoszącą się do linii pliku. Dzięki temu użytkownik otrzymuje komunikat związany z wejściem, a kod elementów nie musi znać kontekstu tekstowego.

Osobną grupę tworzą błędy solvera i macierzy. Solver zgłasza między innymi pustą listę stempli, nieobsługiwaną komendę, niepoprawne parametry analizy czasowej, dywergencję numeryczną oraz brak zbieżności Newtona-Raphsona. Błędy macierzy obejmują na przykład przekroczenie rozmiaru macierzy, niezgodne wymiary albo błąd rozwiązania układu.

5.8 Uwagi implementacyjne

Implementacja w języku Swift pozwala korzystać z typów statycznych, wyjątków oraz bibliotek systemowych dostępnych na platformie macOS. Dla małych macierzy używana jest własna implementacja rozwiązywania układu, natomiast dla większych macierzy można wykorzystać funkcje biblioteki LAPACK dostępne przez framework Accelerate. Takie rozwiązanie pozwala zachować prostotę w przypadkach testowych i jednocześnie pozostawia drogę do obliczeń na większych układach.

Najważniejszą cechą implementacji jest zachowanie prostego kontraktu między warstwami. Parser zwraca stemple i komendę, stemple zwracają macierze lokalne, macierze potrafią dodawać wkłady, a solver wykonuje obliczenia. Dzięki temu każda część programu może być rozwijana i testowana niezależnie.

Rozdział 6

Testy

6.1 Cel testowania

Testy pełnią w projekcie dwie funkcje. Pierwszą jest sprawdzenie poprawności obliczeń wykonywanych przez solver i stemple elementów. Drugą jest kontrola granic odpowiedzialności między częściami programu. Parser powinien wykrywać błędy składniowe i tworzyć poprawne stemple, elementy powinny sprawdzać własne parametry, a solver powinien rozwiązywać układ przygotowany z listy stempli.

Ze względu na charakter aplikacji szczególnie ważne są testy małych układów. Prosty obwód można łatwo przeliczyć ręcznie, dzięki czemu oczekiwany wynik nie jest zależny od samego programu. Takie testy są też wygodne podczas rozwoju, ponieważ szybko wskazują, czy błąd dotyczy parsera, stempla, macierzy czy solvera.

6.2 Zakres testów

Testy zostały podzielone zgodnie z warstwami implementacji. Osobno sprawdzany jest parser, osobno stemple elementów, osobno błędy parametrów, a osobno solver i operacje macierzowe. Dzięki temu pojedynczy test ma jasno określony cel.

Tabela 6.1: Główne grupy testów

Grupa testów	Sprawdzany zakres
Parser	rozpoznawanie elementów, komend analizy, parametrów oraz błędów wejścia
Stemple	poprawność wkładów elementów liniowych, dynamicznych, źródeł i tranzystora BJT
Błędy stempli	reakcja na niepoprawne wartości parametrów elementów
Solver	składanie macierzy globalnej, obliczanie punktu pracy i analiza czasowa
Plotter	przygotowanie serii punktów napięć węzłowych do wykresu
Macierze	dodawanie wartości, dodawanie macierzy i rozwiązywanie układów liniowych

Taki podział ogranicza ryzyko tworzenia testów zbyt szerokich. Jeżeli test parsera sprawdza jednocześnie wynik solvera, trudniej ustalić przyczynę niepowodzenia. W obecnym podejściu parser może być testowany przez typy utworzonych stempli i zgłaszane błędy, natomiast poprawność numeryczna jest sprawdzana w testach solvera.

6.3 Testy parsera

Parser jest testowany zarówno na krótkich opisach zawierających pojedyncze elementy, jak i na pliku zawierającym wszystkie obsługiwane typy elementów. Test z pełnym zestawem elementów sprawdza, czy parser poprawnie rozpoznaje R, C, DCCS, DCVS, ACVS, VCCS, VCVS, CCCS, CCVS oraz BJT. Dodatkowo sprawdzane jest przypisanie dodatkowych indeksów dla elementów wymagających nowych równań.

```
R 1 0 10
DCCS 0 1 2
DCVS 2 0 5
VCCS 1 0 3 0 0.5
VCVS 1 0 4 0 2
CCCS 2 0 5 0 3 1
CCVS 3 0 6 0 4 1
C 1 2 0.0000001
ACVS 7 0 2 1000
BJT 2 3 4 0.000000000001 100 1
.op
```

Listing 6.1: Przykładowy plik używany do sprawdzenia parsera

Testy błędów parsera obejmują między innymi nieznany element, złą liczbę parametrów, niepoprawny typ parametru, niepoprawne parametry stempla, brak komendy, więcej niż jedną komendę analizy, więcej niż jedną komendę `.show` oraz użycie `.show` bez komendy `.tran`.

Każdy z tych przypadków jest istotny z punktu widzenia użytkownika, ponieważ parser powinien wskazać linię, w której znajduje się problem.

Osobno sprawdzana jest komenda `.tran`, w tym poprawność czasu końcowego i kroku czasowego. Dzięki temu format wejściowy jest przygotowany nie tylko dla punktu pracy, ale również dla analizy czasowej. Test parsera nie musi w tym miejscu sprawdzać całej symulacji czasowej; wystarczy potwierdzić, że komenda jest rozpoznawana i przekazywana do dalszej warstwy programu.

6.4 Testy stempli i elementów

Stemple są testowane przez proste obwody, w których wynik można przewidzieć. Dla rezystorów sprawdzane są układy dzielników i połączeń, dla źródeł niezależnych oraz sterowanych sprawdzane są wartości napięć i prądów po rozwiązaniu układu przez solver. Takie testy w praktyce sprawdzają jednocześnie lokalny stempel elementu i jego wpływ na macierz globalną.

Osobne testy dotyczą elementów dodanych dla analizy czasowej i nieliniowej. Dla źródła ACVS sprawdzana jest wartość wymuszenia sinusoidalnego oraz jego wkład do macierzy. Dla kondensatora testowana jest zależność od kroku czasowego i poprzedniego napięcia. Dla tranzystora BJT sprawdzane są macierze tworzone dla danego kontekstu oraz stabilność obliczeń dla typowego punktu pracy.

W testach elementów nie ma potrzeby sprawdzania szczegółów wewnętrznego numerowania wszystkich wierszy macierzy, jeśli ten mechanizm jest już testowany w parserze i solverze. Ważniejsze jest potwierdzenie, że gotowy układ daje oczekiwany wynik. Dzięki temu testy opisują zachowanie programu, a nie tylko szczegóły implementacji.

6.5 Testy błędów stempli

Błędy parametrów stempli są testowane oddzielnie. Dotyczy to sytuacji, w których dane mają poprawny typ, ale nie mogą zostać użyte przez element. Przykładem jest ujemny numer węzła albo wartość nieskończona podana jako parametr.

Wspólny typ `StampParameterError` obejmuje trzy sytuacje: ujemny indeks węzła, niepoprawny parametr elementu oraz błąd programistyczny związany z niepoprawnym wewnętrznym indeksem. Testy obejmują między innymi zerową lub nieskończoną wartość parametrów elementów, niepoprawne parametry źródła sinusoidalnego, kondensatora oraz tranzystora.

Oddzielenie tych testów od testów parsera jest korzystne. Parser odpowiada za strukturę pliku, natomiast element odpowiada za sensowność własnych parametrów. Parser może później przekształcić błąd stempla w komunikat zawierający numer linii. Sama decyzja o tym, czy parametr elementu jest poprawny, pozostaje w klasie elementu.

6.6 Testy solvera i macierzy

Testy solvera sprawdzają, czy lista stempli jest poprawnie składana do jednej macierzy globalnej oraz czy wynik rozwiązania odpowiada wartościom oczekiwany dla prostych obwodów. Przykładem jest obwód z idealnym źródłem napięcia i rezystorem, w którym napięcie węzła można odczytać bezpośrednio z wartości źródła.

Dla analizy czasowej sprawdzany jest wynik `TransientResult`: liczba próbek, wartości czasu oraz napięcie w prostym układzie ze źródłem sinusoidalnym i rezystorem. Testy błędów solvera obejmują pustą listę stempli, nieobsługiwaną komendę, błędne parametry analizy czasowej, dywergencję numeryczną oraz brak zbieżności metody Newtona-Raphsona.

6.7 Testy wizualizacji

Moduł `Plotter` jest testowany bez uruchamiania interfejsu użytkownika. Test otrzymuje przygotowany wynik analizy czasowej i sprawdza, czy dla wskazanych węzłów powstają poprawne serie punktów. Osobno sprawdzane są sytuacje błędne, na przykład odwołanie do nieistniejącego węzła albo niespójna liczba próbek czasu i rozwiązań.

Takie podejście pozwala testować logikę przygotowania danych do wykresu bez kosztownego uruchamiania testów interfejsu. Warstwa graficzna pozostaje prosta, ponieważ otrzymuje już gotowe serie punktów.

Testy macierzy sprawdzają operacje niższego poziomu: dodawanie pojedynczych wartości, dodawanie jednej macierzy do drugiej oraz rozwiązywanie układów równań. Jest to ważne, ponieważ błędy w indeksowaniu macierzy prowadziłyby do pozornie poprawnych, ale numerycznie błędnych wyników symulacji.

6.8 Znaczenie testów dla rozwoju programu

Przyjęty zestaw testów ułatwia rozszerzanie aplikacji o nowe elementy i nowe rodzaje analizy. Dodanie kolejnego elementu powinno wiązać się z testem parsera, testem poprawnego stempla oraz testem przynajmniej jednego prostego obwodu z tym elementem. W przypadku nowych komend analizy potrzebne są testy rozpoznawania komendy i testy solvera.

Testy stanowią również dokumentację zachowania programu. Pokazują, jakie elementy są obsługiwane, jaki format danych wejściowych jest poprawny i jakie błędy powinny zostać zgłoszone w niepoprawnych przypadkach. Jest to szczególnie przydatne w projekcie rozwijanym etapami, gdzie kolejne funkcje są dodawane bez zmiany podstawowej architektury.

Rozdział 7

Podsumowanie

7.1 Wnioski

W pracy przedstawiono projekt i implementację dedykowanej aplikacji do analizy analogowych układów elektronicznych. Podstawą obliczeń jest zmodyfikowana metoda potencjałów węzłowych, która dobrze pasuje do programowej reprezentacji obwodu przez stemple elementów. Każdy element może zostać opisany lokalnie, a następnie jego wkład jest dodawany do globalnego układu równań.

Zrealizowana aplikacja pokazuje pełny przepływ danych od tekstowego opisu układu do wyniku analizy. Użytkownik może wpisać netlistę w głównym oknie programu albo przeciągnąć plik tekstowy. Parser odczytuje opis, rozpoznaje obsługiwane elementy, komendy `.op` i `.tran` oraz listę węzłów wskazanych przez `.show`. Następnie tworzy listę stempli i przekazuje ją do solvera. Solver składa macierz globalną i wyznacza wartości niewiadomych, a dla analizy czasowej przygotowuje kolejne próbki rozwiązania. Wyniki mogą zostać następnie przedstawione jako wartości punktu pracy albo w postaci wykresu napięć wybranych węzłów. Taki podział odpowiedzialności okazał się wygodny zarówno podczas implementacji, jak i podczas pisania testów.

Istotnym rezultatem pracy jest również przygotowanie struktury programu do dalszego rozwoju. Obsługa elementów nie jest zaszyta bezpośrednio w solverze, lecz wynika z interfejsu stempla. Dzięki temu dodanie kolejnego elementu polega głównie na zdefiniowaniu jego wkładu macierzowego i rozszerzeniu parsera o nowy typ. Podobnie komendy analizy są przekazywane jako osobna informacja, co pozwala rozwijać kolejne tryby obliczeń bez przebudowy formatu danych wewnętrznych.

Wybór języka Swift i platformy macOS pozwolił wykorzystać typowanie statyczne, mechanizmy obsługi błędów oraz biblioteki numeryczne dostępne w systemie. Jednocześnie implementacja pozostała czytelna, ponieważ podstawowe pojęcia programu odpowiadają pojęciom teoretycznym: element, stempel, macierz, wektor wymuszeń i solver.

7.2 Ocena przyjętego podejścia

Największą zaletą przyjętego podejścia jest modularność. Parser, elementy, macierze i solver mogą być testowane osobno, a ich interfejsy są stosunkowo proste. Ogranicza to liczbę zależności między częściami programu i zmniejsza ryzyko, że zmiana jednego elementu wymusi przebudowę całej aplikacji.

Drugą zaletą jest zgodność implementacji z metodą ZMPW. W metodzie tej globalny układ równań powstaje przez sumowanie lokalnych wkładów elementów. Tę samą zasadę wykorzystuje program: każdy stempel zwraca własną macierz i wektor, a solver dodaje je do macierzy globalnej. Dzięki temu kod nie wymaga osobnych algorytmów składania dla każdego typu obwodu.

Ograniczeniem obecnego rozwiązania jest uproszczony format wejściowy. Netlista jest czytelna i wystarczająca dla obsługiwanych elementów, ale nie zawiera jeszcze bardziej rozbudowanych konstrukcji znanych z pełnych symulatorów SPICE, takich jak modele elementów, parametry nazwane, podukłady czy złożone źródła zależne od czasu. Jest to świadomy kompromis, ponieważ głównym celem pracy było pokazanie działania metody i architektury programu, a nie odtworzenie pełnego standardu SPICE.

7.3 Możliwe kierunki rozwoju

Dalszy rozwój programu może obejmować kilka obszarów. Pierwszym z nich jest rozszerzenie biblioteki elementów. Program obsługuje już rezystory, kondensatory, wybrane źródła niezależne i sterowane oraz uproszczony model tranzystora bipolarnego. Naturalnym kolejnym krokiem są cewki, diody, dokładniejsze modele tranzystorów oraz parametry modeli definiowane w pliku wejściowym.

Drugim kierunkiem jest rozbudowa analizy czasowej. Obecna architektura pozwala używać tych samych mechanizmów składania macierzy w kolejnych krokach czasowych, a komenda `.tran` przyjmuje czas końcowy i krok czasowy. W przyszłości można dodać jawnie definiowane warunki początkowe, adaptacyjny dobór kroku oraz różne metody całkowania numerycznego, na przykład metodę Geara. Warto również rozwinąć stabilizację metody Newtona-Raphsona przez tłumienie iteracji, ograniczanie zmian napięć w kolejnych iteracjach i bardziej rozbudowane kryteria zbieżności.

Trzecim obszarem jest rozwój formatu wejściowego. Warto dodać obsługę komentarzy, jednostek z przedrostkami, parametrów nazwanych oraz bardziej opisowych komunikatów błędów. Przy większych projektach przydatne byłyby także podukłady i modele elementów definiowane w osobnych sekcjach pliku.

Czwartym kierunkiem jest dalsza rozbudowa warstwy użytkownika. Program zawiera już okno edycji netlisty, obsługę przeciągania plików tekstowych, uruchamianie analizy oraz podstawową prezentację wyników. Dla analizy czasowej szczególnie przydatne byłyby eksport da-

nych do plików, powiększanie i przesuwanie wykresu, możliwość porównania kilku przebiegów oraz wybór obserwowanych wielkości bez ręcznej edycji netlisty.

7.4 Zakończenie

Praca potwierdza, że zmodyfikowana metoda potencjałów węzłowych jest dobrą podstawą do budowy symulatora układów analogowych. Metoda zapewnia przejrzyste przejście od schematu obwodu do układu równań, a reprezentacja przez stemple dobrze przekłada się na strukturę programu.

Zaimplementowany projekt stanowi punkt wyjścia do dalszego rozwoju. Obsługuje podstawowe elementy liniowe, element dynamiczny, wybrane źródła niezależne i sterowane, uproszczony model tranzystora BJT, parser tekstowego opisu obwodu, mechanizm błędów, solver punktu pracy, analizę czasową, graficzne wprowadzanie netlisty oraz podstawową wizualizację wyników. Przykład wtórnika emiterowego pokazuje, że program może zostać użyty nie tylko do prostych układów liniowych, ale również do symulacji układu zawierającego element dynamiczny i nieliniowy. Najważniejsze jest jednak to, że architektura programu pozwala dodawać kolejne elementy i analizy bez porzucania przyjętego modelu obliczeń.

Bibliografia

- [1] L. W. Nagel i D. O. Pederson, „SPICE: Simulation Program with Integrated Circuit Emphasis”, University of California, Berkeley, sprawozdanie techniczne UCB/ERL M382, 1973. URL: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1973/22871.html>
- [2] L. W. Nagel, „SPICE2: A Computer Program to Simulate Semiconductor Circuits”, PhD thesis, University of California, Berkeley, 1975. URL: <https://www2.eecs.berkeley.edu/Pubs/TechRpts/1975/9602.html>
- [3] K. S. Kundert, *The Designer's Guide to SPICE and Spectre*. Springer, 1995. DOI: 10.1007/b101824
- [4] C.-W. Ho, A. E. Ruehli i P. A. Brennan, „The Modified Nodal Approach to Network Analysis”, *IEEE Transactions on Circuits and Systems*, t. 22, nr 6, s. 504–509, 1975. DOI: 10.1109/TCS.1975.1084079
- [5] J. Vlach i K. Singhal, *Computer Methods for Circuit Analysis and Design*. New York: Van Nostrand Reinhold, 1994, ISBN: 0-442-01194-6.
- [6] Apple Inc., *The Swift Programming Language*, 2026. URL: <https://docs.swift.org/swift-book/documentation/the-swift-programming-language/>
- [7] L. O. Chua i P.-M. Lin, *Computer-Aided Analysis of Electronic Circuits: Algorithms and Computational Techniques*. Prentice-Hall, 1975.
- [8] A. Quarteroni, R. Sacco i F. Saleri, *Numerical Mathematics*, 2 wyd. Springer, 2007.
- [9] Apple Inc., *SwiftUI*, 2026. URL: <https://developer.apple.com/documentation/swiftui>
- [10] Apple Inc., *Uniform Type Identifiers*, 2026. URL: <https://developer.apple.com/documentation/uniformtypeidentifiers>